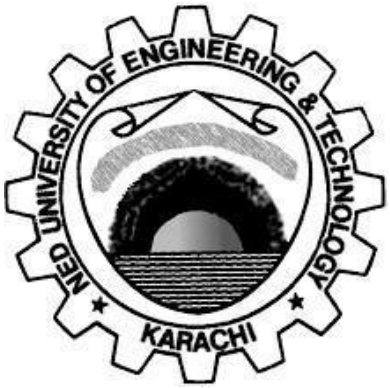
Complex Computing Problem Report

CT-486: Network and Information Security

Final Year - Cyber Security

Batch: 2023



Group Members:

Mr. Ameem Akhtar ..... **(CR-23013)**

Mr. Minhal Rizvi ..... **(CR-23017)**

Mr. Muhammad Usman ..... **(CR-23038)**

Mr. Maaz Wasim ..... **(CR-23041)**

Submitted to: Prof. Dr. Shariq Mahmood Khan (Professor, Department of CS & IT)

Submission Date: November 02, 2026

1. Problem Statement and Design Overview

The brief asks for a custom cipher built from at least two classical ciphers, in at least two stages that each use a different technique, with a key of at least 10 characters; a program that encrypts and decrypts A-Z text of different lengths; an attack that breaks the cipher; and an analysis of its efficiency against a single-stage Shift cipher.

This project combines a **Vigenere cipher** (polyalphabetic substitution) with a **2 × 2 Hill cipher** (polygraphic matrix transformation over Z_{26}). It is implemented in Python (`custom_cipher.py`), and the attack is a **known-plaintext attack** (`attack.py`). The main result is that the second stage hides letter frequencies well but adds little real security: both stages are linear, and the combination is broken in about a second by an attacker who knows roughly 40 letters of a message.

Mathematical Definition

Letters map to A=0, ..., Z=25. The padded plaintext P is encrypted with a Vigenere key K_{vig} of length $L \geq 10$ and an invertible matrix K_{hill} = [[a, b], [c, d]]:

```
Stage 1 (Vigenere):  C1[i] = (P[i] + Kvig[i mod L]) mod 26
Stage 2 (Hill):      [C[2j], C[2j+1]]^T = Khill * [C1[2j], C1[2j+1]]^T mod 26
```

Stage 2 can be undone only if gcd(det K_{hill}, 26) = 1, that is, the determinant $\Delta = (ad - bc) \bmod 26$ is neither even nor a multiple of 13. Decryption uses $K_{hill}^{-1} = \Delta^{-1} \cdot [[d, -b], [-c, a]] \bmod 26$. There are $|GL(2, Z_{26})| = 157{,}248$ such matrices.

How the Classical Techniques Are Combined

- **Keys.** The cipher takes two keys, each at least 10 letters after removing non-letters; shorter keys are rejected. The Vigenere key is used letter by letter. The Hill phrase is reduced to a matrix by adding each of its letters, in turn, into entry a, b, c or d mod 26, so every letter of the phrase affects the matrix. If the determinant is not invertible, entry d is incremented (carrying into c, b, a) until it is.
- **Order of the stages.** Vigenere runs first because it needs to act on individual letters at known positions. Its output is cut into pairs and each pair is multiplied by the Hill matrix, so each ciphertext letter depends on two Vigenere-shifted letters and on the key letters at both positions.
- **What each stage contributes.** Vigenere replaces one fixed alphabet with L alphabets, which flattens the letter histogram. Hill then mixes neighbouring letters, which hides the repeating key alphabet from column-by-column Vigenere analysis, since a ciphertext letter no longer belongs to a single Caesar alphabet.
- **Padding.** Hill needs an even number of letters. An odd-length message gets 'X' and an even-length message gets 'XA'. Padding is always added so that decryption can remove it without guessing: a final 'X' means one padding letter, anything else means two. A message genuinely ending in X, such as FOXX, therefore decrypts exactly.

Worked Example

```
Keys:        Kvig = CRYPTOGRAPHYKEY (L = 15), Hill phrase = HILLCIPHERKEY
Matrix:      Khill = [[11, 7], [10, 23]]
Plaintext:   ATTACKATDAWN
Padded:      ATTACKATDAWNXA
Stage 1:     CKRPVYGKDPDLHE
Stage 2:     OQGVJIGEILGXBG   (final ciphertext)
Decrypted:   ATTACKATDAWN
```

2. Encryption and Decryption Algorithms

Algorithm 1: Encryption

```
Input:  Plaintext P, Vigenere key Kvig (|Kvig| >= 10), Hill phrase H (|H| >= 10)
Output: Ciphertext C
 1: P <- keep only letters A-Z of UpperCase(P)
 2: if P is empty then return empty string
 3: if |P| is odd then P <- P + 'X' else P <- P + 'XA'
 4: Khill <- DeriveMatrix(H)
 5: for i = 0 to |P| - 1 do
 6:     C1[i] <- (P[i] + Kvig[i mod |Kvig|]) mod 26
 7: for j = 0 to |P|/2 - 1 do
 8:     C[2j]   <- (a * C1[2j] + b * C1[2j+1]) mod 26
 9:     C[2j+1] <- (c * C1[2j] + d * C1[2j+1]) mod 26
10: return C
```

Algorithm 2: Decryption

```
Input:  Ciphertext C, Vigenere key Kvig, Hill phrase H
Output: Original plaintext P
 1: if |C| is odd then reject as invalid
 2: Khill <- DeriveMatrix(H)
 3: Kinv <- ModInverse(det Khill, 26) * [[d, -b], [-c, a]] mod 26
 4: for j = 0 to |C|/2 - 1 do
 5:     C1[2j]   <- (Kinv.a * C[2j] + Kinv.b * C[2j+1]) mod 26
 6:     C1[2j+1] <- (Kinv.c * C[2j] + Kinv.d * C[2j+1]) mod 26
 7: for i = 0 to |C| - 1 do
 8:     P[i] <- (C1[i] - Kvig[i mod |Kvig|]) mod 26
 9: if P ends in 'X' then remove 1 letter else remove 2 letters
10: return P
```

DeriveMatrix(H)

```
1: [a, b, c, d] <- [0, 0, 0, 0]
2: for i = 0 to |H| - 1 do  entry[i mod 4] <- (entry[i mod 4] + H[i]) mod 26
3: while gcd(ad - bc, 26) != 1 do  increment d, carrying into c, b, a at 26
4: return [[a, b], [c, d]]
```

The unit tests in `test_suite.py` check exact round trips for every length from 1 to 26, a 380-letter message, and padding-like endings such as X, XX, XA and FOXX.

3. Security Analysis

Key space. With L = 10 there are $26^{10} \times 157{,}248 \approx 2.2 \times 10^{19}$ key pairs, against 26 for the Shift cipher, so exhaustive search of both keys together is impractical. This number overstates the security, because the attack in Section 4 never searches the Vigenere key: it searches the matrices alone and solves for the Vigenere key directly.

Structure. Substituting Stage 1 into Stage 2 gives $C_j = K_{hill} \cdot P_j + K_{hill} \cdot V_j \bmod 26$, where V_j is the pair of key letters at block j. The whole cipher is therefore a single affine Hill cipher whose constant term repeats with the key. Composing two linear stages does not create a stronger kind of function, and that is the weakness the attack exploits.

Confusion and diffusion. Vigenere provides confusion: the same letter encrypts differently at different positions. Diffusion is limited to one pair. Changing one plaintext letter changed **1.77** ciphertext letters on average and never more than 2, whereas a modern block cipher changes about half of its output.

Frequency analysis. Letter counts of each ciphertext were compared with the expected English distribution, using the Index of Coincidence $IoC = \sum f_i(f_i - 1) / [N(N - 1)]$ and the chi-squared distance from English. Figures are averages over 30 random key sets and 5,000-letter passages of English technical documentation, which runs slightly above the textbook English IoC of 0.0667.

Text	Mean IoC	Std. dev.	Chi-squared vs English

Plaintext	0.0703	0.0024	534.0
Shift ciphertext	0.0703	0.0024	61,406.1
Hill only	0.0445	0.0042	29,986.2
Vigenere only	0.0407	0.0012	29,234.7
Custom (Vigenere + Hill)	0.0391	0.0006	29,480.5

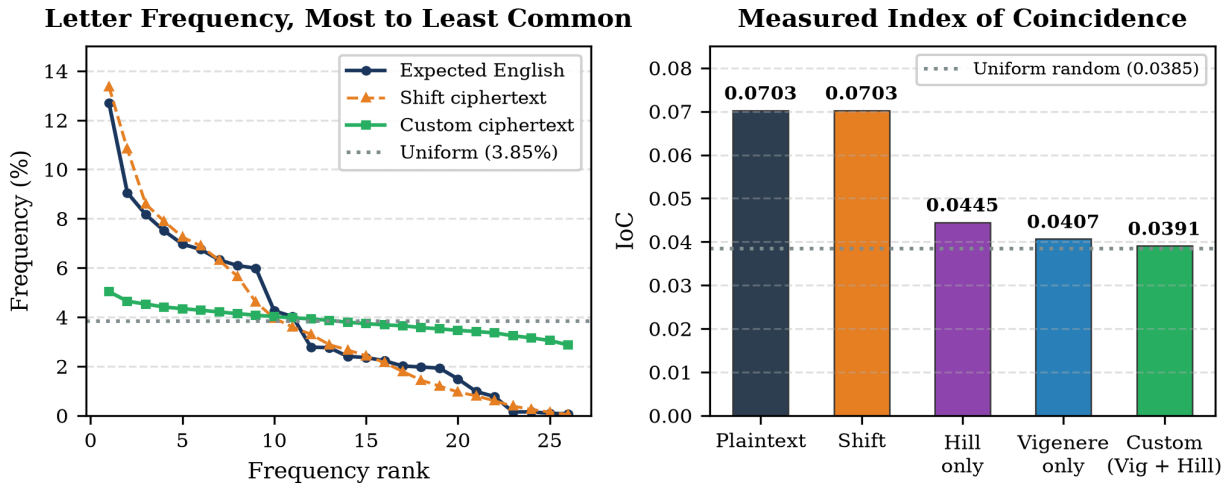


Figure 1: Measured letter frequency of each ciphertext against expected English, and measured IoC.

The Shift cipher has exactly the plaintext's IoC (0.0703) because it only renames letters. The custom cipher reaches 0.0391, close to the uniform value of 0.0385, and its histogram no longer resembles English. The chi-squared value shows only that the ciphertext is far from English. Frequency analysis on its own therefore does not recover the key here, so it serves as a measure of how well the statistics are hidden; the cipher is broken by the attack below.

4. Attack Method and Results

Scenario. The attacker has the whole ciphertext and the plaintext of its first M letters, such as a predictable message header, and knows the algorithm and that keys have at least 10 letters. The attacker does **not** know either key or the Vigenere key length.

Algorithm 3: Known-Plaintext Attack

```
Input:  Ciphertext C, known plaintext P[0..M-1]
Output: Khill, Kvig and the full plaintext, or failure
 1: Pool <- all 157,248 invertible 2x2 matrices mod 26 with their inverses
 2: for L = 10 to M - 20 do                        // candidate key lengths
 3:     for each matrix K in Pool do
 4:         for each known pair j do
 5:             [x1, x2]^T <- Kinv * [C[2j], C[2j+1]]^T mod 26
 6:             k <- (x - P) mod 26 for both letters
 7:             if k disagrees with a key letter already set at i mod L then reject K
 8:         if K was not rejected then decrypt all of C with (K, k); return
 9: return failure
```

Why it works. Once the correct matrix is guessed, Stage 2 is undone and Stage 1 reduces to k[i] = x[i] - P[i], which directly reveals the Vigenere key. A wrong matrix produces a stream that does not repeat every L letters. The check exploits exactly the two properties identified in Section 3: linearity and the repeating key. Known pairs are tested grouped by key position, so a wrong matrix is usually rejected after a few checks.

How much known text is needed. At least L letters are needed to see every key letter, and extra letters are needed to reject wrong matrices. Because 26 = 2 × 13, a matrix that is correct mod 13 and wrong only mod 2 passes each check about half the time. In a separate experiment (`false_match_experiment.py`) with L = 20, 8 extra letters returned a wrong key in 14 of

150 runs and 20 extra letters in 0 of 150, so the attack requires $M \geq L + 20$ and reports failure otherwise.

Results. Each row is 20 trials, each with new random keys (Vigenere key of 10 to 20 letters) and a new random English passage. A trial counts as a success only if the complete message is recovered exactly. Time excludes the one-off construction of the matrix pool, which took 350 ms.

Table 1: Success and effort by ciphertext length (first 25% of the message known)

Ciphertext length	Known letters	Success rate	Average time (ms)	Worst time (ms)	Average matrices tried
50	12	0.0%	0.0	0.0	0
100	25	0.0%	0.0	0.1	0
150	37	80.0%	804.6	1742.5	650,590
200	50	100.0%	1292.5	2339.3	934,022
500	125	100.0%	1194.1	2307.1	862,325
1,000	250	100.0%	1179.8	2817.9	866,487
5,000	1,250	100.0%	1184.2	2440.0	822,389
10,000	2,500	100.0%	1225.3	2735.2	866,181

Table 2: Success and effort by amount of known plaintext (1,000-letter ciphertext)

Known letters	Success rate	Average time (ms)	Worst time (ms)	Average matrices tried
20	0.0%	0.2	0.3	0
30	0.0%	208.9	278.8	157,248
34	25.0%	880.3	1411.5	683,240
36	65.0%	1200.0	2014.7	808,350
40	100.0%	1105.9	2321.7	840,348
50	100.0%	1244.3	2193.8	952,363
60	100.0%	1502.4	2517.4	1,091,306
100	100.0%	1204.7	2356.3	916,394

Success is decided by the amount of known text, not by the ciphertext length as such. Short messages fail in Table 1 only because 25% of them is too few letters. With 40 or more known letters, enough for the longest key tested, recovery was exact in every trial. When recovery succeeded, the attack averaged about 1215 ms and at most 2818 ms, and its cost did not grow with the ciphertext length.

5. Performance and Efficiency

Time complexity, for a message of N letters:

- **Encryption:** Vigenere does N additions and Hill does N/2 matrix multiplications of constant size, so the total is **O(N)**.
- **Decryption:** inverting a 2 × 2 matrix takes constant time, followed by the same two passes in reverse, so the total is **O(N)**.
- **Shift cipher:** one addition per letter, also **O(N)**. All three use O(N) space. The custom cipher is slower by a constant factor only.
- **Attack:** the matrix pool is a one-off $26^4 = 456{,}976$ determinant checks. The search is $O(T \cdot |GL| \cdot c)$, where T is the number of key lengths tried, $|GL| = 157{,}248$ and c is the average number of checks before rejection, which is small and in the worst case M. Recovering the message afterwards is O(N).

Table 3: Measured cipher time in ms (average of 50 runs, Python 3.14.4)

Length	Custom encrypt	Custom decrypt	Shift encrypt	Shift decrypt	Encrypt overhead	Custom letters/s
100	0.0677	0.0508	0.0222	0.0225	3.05x	1,477,105
500	0.2882	0.2871	0.1182	0.1176	2.44x	1,734,906
1,000	0.4993	0.6358	0.2196	0.2153	2.27x	2,002,804
5,000	2.4265	2.2959	1.0988	1.1122	2.21x	2,060,581
10,000	4.7058	4.7004	2.5343	2.3451	1.86x	2,125,037

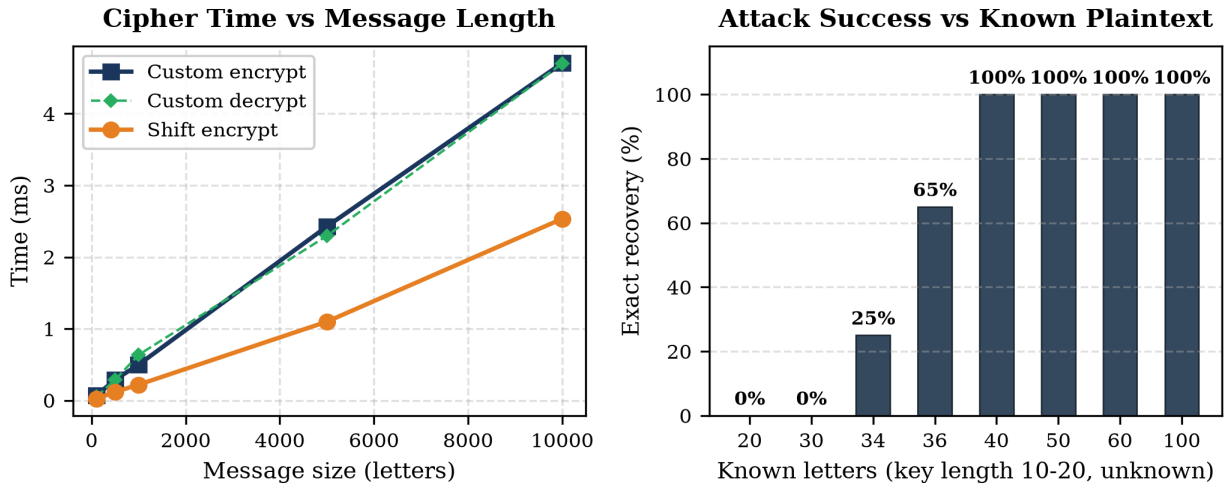


Figure 2: Measured cipher time against message length, and attack success against known plaintext.

Time grows linearly with length for both ciphers, as the analysis predicts. Encrypting 1,000 letters took 0.499 ms with the custom cipher and 0.220 ms with the Shift cipher. The two-stage design costs 1.9x to 3.0x the time of the single-stage cipher, and in return it hides letter frequencies that the Shift cipher exposes completely. These are timings of interpreted Python on one machine, so the ratios are more meaningful than the absolute values.

6. Improving Security Without Significantly Increasing Complexity

The attack depends on linearity and on the repeating key, so useful changes must remove one of these. Each option below stays O(N). They would raise the attacker's cost but were not implemented or measured here, and none would make the cipher secure by modern standards.

1. **Keyed non-linear substitution between the stages.** Pass each Vigenere output letter through a secret permutation of the alphabet, one table lookup per letter. The relation k[i] = x[i] - P[i] then no longer holds, so the key letters cannot be read off after guessing the matrix.
2. **Keyed columnar transposition between the stages.** Reordering letters by a secret permutation separates known plaintext from the ciphertext positions it produces, so the attacker must also search the permutation.
3. **A non-repeating keystream.** Derive a different Vigenere key for each message from the main key and a per-message value. The periodicity check that rejects wrong matrices then has nothing to test.

Two changes that seem helpful are weak on their own. A longer Vigenere key only raises the known text needed ($M \geq L + 20$). A larger Hill matrix leaves the system linear: subtracting blocks a whole number of key periods apart cancels the Vigenere term and gives $\Delta C = K_{hill} \cdot \Delta P$, which can be solved with linear algebra rather than search.

7. Code Submission

The source code, tests and measurement scripts are at: **https://github.com/ameemakhtr/NIS_CCP**. Running `python benchmark.py` reproduces `results.json`, from which every table and figure above is generated.